07 · Moderne CPU-Architekturen

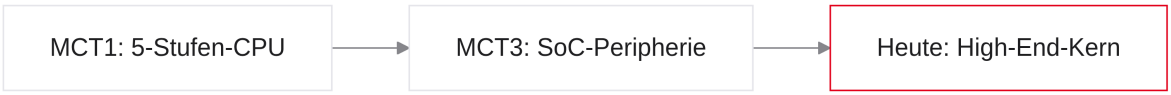
Mikrocomputertechnik 3 · Von Pipelining-Limits über Out-of-Order zur Branch Prediction

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 67

Willkommen zu Block 7!

- Willkommen zur siebten Vorlesung in Mikrocomputertechnik 3
- Bisher: RAM, Caches, Interrupts, Busse, DMA — das SoC ist ausgestattet
- Peripherie und Transfers können autonom laufen; der Engpass wandert zurück zum Kern
- Heute: wie High-End-CPUs intern mehr Rechenleistung aus demselben Takt holen



Notizen

Willkommen zu Block 7 knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. DMA entlastet Kopierarbeit, beseitigt aber nicht jeden Speicherwartezustand. Heute geht es um mehrere Befehle im Kern. Nicht jede CPU derselben Klasse ist gleich breit oder OoO. Willkommen zur siebten Vorlesung in Mikrocomputertechnik 3. Bisher: RAM, Caches, Interrupts, Busse, DMA — das SoC ist ausgestattet. Verständnischeck: Wartet die CPU trotz DMA nie? Nein, eigene Loads können warten. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 2 / 67

Anknüpfung: DMA-Lehrszenario

Kurz-Recap aus Block 6 (Vertiefung im Übungsblatt):

- DMA kopiert Arrays autonom; Polling blockiert die CPU stark
- Ein Heartbeat aus `main()` kann weiterlaufen. Das beweist keine Last von null
- Bus-Mastering und Cycle Stealing: DMA teilt sich den Bus mit der CPU
- Folge: weniger ALU-Zeit für stupides Kopieren — mehr Zeit für echte Berechnung

```
/* Konzept - Adressen/Bits fiktiv bzw. Datenblatt */
DMA_CH1->SRC_ADDR = (uint32_t)text_array;
DMA_CH1->DST_ADDR = (uint32_t)&UART_TX;
DMA_CH1->CONTROL = DMA_ENABLE;  /* fiktiv, kein blindes |= */
```

Notizen

Anknüpfung: DMA-Lehrszenario knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Die drei Zuweisungen sind ein unvollständiger Lehrausschnitt: Quelle, festes UART-Ziel, Enable. Länge, Breite, Inkrement, Request, Besitz und Abschluss fehlen. Der uint32_t-Cast ist keine DMA-Adresse. Ein Heartbeat zeigt Fortschritt, keine Null-Last. DMA kopiert Arrays autonom; Polling blockiert die CPU stark. Ein Heartbeat aus `main()` kann weiterlaufen. Das beweist keine Last von null. Verständnischeck: Reichen Quelle, Ziel und Enable? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 3 / 67

Zurück in die CPU

- Fokus: Ausführungsgeschwindigkeit im Kern
- Weg von Systembus, MMIO und Peripherie-IRQs
- Fokus auf den Kern; Cache-Misses bleiben in den Zeitbeispielen ausdrücklich drin
- Metriken und Hardware-Parallelität innerhalb eines Kerns

Notizen

Zurück in die CPU knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Der Fokus liegt auf dem Kern, Cache-Misses bleiben in den Zeitbeispielen. Assembler nennt einen ISA-Befehl. Pipeline und physische Register sind Implementierung. Gleiche Programme können verschieden viele Takte brauchen. Fokus: Ausführungsgeschwindigkeit im Kern. Weg von Systembus, MMIO und Peripherie-IRQs. Verständnischeck: Warum später ein Cache-Miss? Er bestimmt, wann der Operand bereit ist. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Refresher MCT1: 5-Stufen-Pipeline

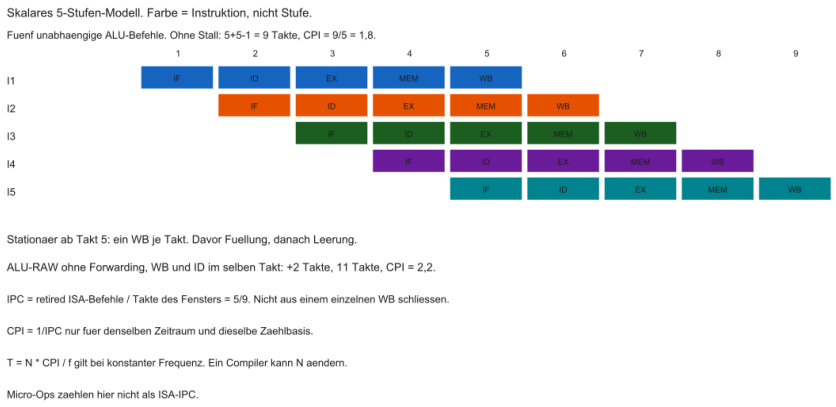


Abbildung 1: Fünf ALU-Befehle, neun Takte, CPI 1,8. Benachbarter ALU-RAW ohne Forwarding: elf Takte, CPI 2,2.

Klassische RISC-Pipeline (Harris/Harris):

Stufe	Name	Aufgabe
IF	Instruction Fetch	Befehl holen
ID	Decode	Dekodieren, Register lesen
EX	Execute	ALU
MEM	Memory	Datenspeicher
WB	Write Back	Ergebnis ins Registerfile

Nach dem Füllen strebt der Durchsatz gegen ein Retire je Takt. Fünf Befehle brauchen im Modell ohne Stall $N + k - 1 = 9$ Takte, mittleres CPI $= 1{,}8$.

Notizen

Refresher MCT1: 5-Stufen-Pipeline knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. IF holt, ID dekodiert und liest, EX rechnet, MEM ist der Speicherplatz, WB schreibt zurück. Fünf unabhängige ALU-Befehle brauchen neun Takte: CPI 9/5 = 1,8, IPC 5/9. Stationär ist ein Befehl je Takt. Ohne Forwarding schreibt I1 in WB Takt 5, I2 liest erst dann und startet EX in Takt 6. Das sind zwei Zusatzstalls, elf Takte, CPI 2,2. Verständnischeck: Warum CPI 1,8 statt 1? Das kurze Fenster enthält Füllen und Leeren. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 5 / 67

Metriken: IPC und CPI

- IPC: retired ISA-Instruktionen pro Core-Takt im betrachteten Fenster
- Execution-Completion und Retire können in verschiedenen Takten liegen
- Micro-Ops sind keine ISA-IPC
- CPI = Takte / retired Instruktionen. CPI und IPC sind nur dann Kehrwerte
- Laufzeit bei konstanter Frequenz f im selben Fenster:

$$T = N \times \text{CPI} / f$$

Notizen

Metriken: IPC und CPI knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. IPC zählt retired ISA-Instruktionen je Takt, CPI die Takte je retired Instruktion, nur im selben Fenster Kehrwerte. Completion ist nicht Retire. 100 Befehle mit CPI 1 brauchen 100 Takte, 150 mit CPI 0,8 brauchen 120. Höheres IPC kann trotzdem länger dauern. IPC: retired ISA-Instruktionen pro Core-Takt im betrachteten Fenster. Execution-Completion und Retire können in verschiedenen Takten liegen. Verständnischeck: Sind CPI und IPC immer Kehrwerte? Nur bei gleicher Basis und gleichem Fenster. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 6 / 67

Limit der skalaren Pipeline

- Ideal ohne Stalls/Hazards/Misses: IPC $= 1$, CPI $= 1$
- Eine skalare Ein-Issue-Pipeline: strukturell kein CPI < 1
- Ein Pfad → Befehle im Gänsemarsch
- Realität oft schlechter (CPI > 1) durch Bubbles und Hazards

Notizen

Limit der skalaren Pipeline knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Eine skalare Pipeline überlappt Stufen, startet hier aber höchstens einen neuen Befehl je Takt. Ideal 1 gilt stationär ohne Warten. Ein späterer Retire-Burst ist kein dauerhaftes IPC über 1. Ideal ohne Stalls/Hazards/Misses: IPC $= 1$, CPI $= 1$. Eine skalare Ein-Issue-Pipeline: strukturell kein CPI < 1. Verständnischeck: Laufen bei IPC 1 nur einzelne Befehle? Nein, mehrere Stufen sind zugleich belegt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 7 / 67

Die Taktraten-Mauer (Power Wall)

- Idee der 2000er: Frequenz massiv erhöhen (GHz-Wettrennen)
- Vereinfacht $P_{dyn} \approx \alpha C V^2 f$. Leckleistung ist davon getrennt
- Keine feste Frequenzgrenze aus dieser Formel. Breitere Kerne brauchen mehr Ports, Scheduler und Bypass
- CPI verbessern ist ein Weg, kein kostenloser Ersatz für Takt

Notizen

Die Taktraten-Mauer (Power Wall) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. P_dyn ist etwa alpha mal C mal V quadrat mal f. C ist Kapazität. Bei festem alpha und C ergibt 1,5-fache Frequenz und 1,1-fache Spannung den Faktor 1,815 nur für die dynamische Leistung. Leckstrom ist getrennt. Die Formel liefert keine feste Gigahertz-Grenze. Idee der 2000er: Frequenz massiv erhöhen (GHz-Wettrennen). Vereinfacht $P_{dyn} \approx \alpha C V^2 f$. Leckleistung ist davon getrennt. Verständnischeck: Liefert die Formel eine maximale Taktzahl? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 8 / 67

Die Leitfrage des Tages

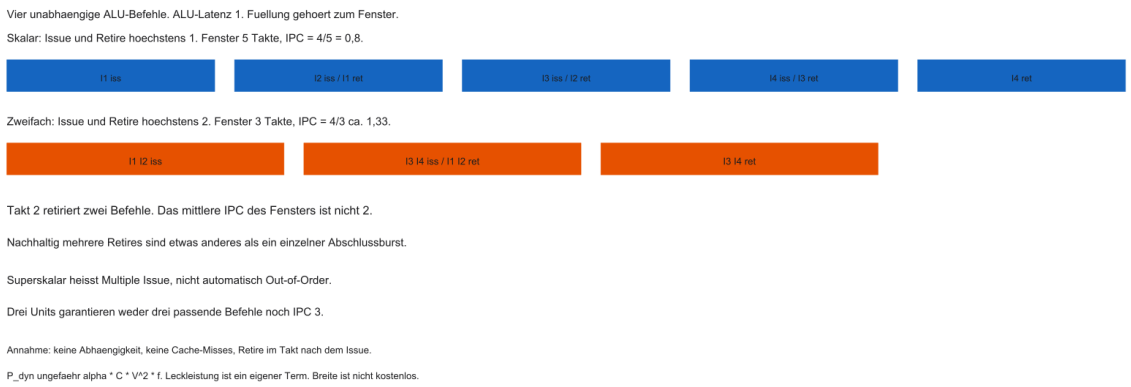


Abbildung 2: Skalar 4/5 = 0,8. Zweifach 4/3 ≈ 1,33. Ein Doppeltakt ist nicht das mittlere IPC.

- Taktfrequenz: physikalisch begrenzt
- Skalare Pipeline: Ideal CPI $= 1$ (mit Stalls oft > 1)
- Wie kann der Kern langfristig mehr als eine ISA-Instruktion je Takt retiren?
- Antwort: Parallelität *innerhalb* des Kerns (ILP ausnutzen)

Das Zeitdiagramm oben ersetzt die Ein-Takt-Skizze. IPC wird über das ganze Fenster gerechnet.

Notizen

Die Leitfrage des Tages knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Im verkürzten Modell M2 sind vier unabhängige Befehle bereit. Skalar: Issue in 1 bis 4, Retire in 2 bis 5, IPC 4/5 = 0,8. Zweifach: zwei Issue-Paare, Retire in Takt 2 und 3, IPC 4/3. Ein Doppeltakt ist nicht das mittlere IPC 2. Dieses Fenster ist nicht das Fünf-Stufen-Modell. Taktfrequenz: physikalisch begrenzt. Skalare Pipeline: Ideal CPI $= 1$ (mit Stalls oft > 1). Verständnischeck: Warum 4/3 und nicht 2? Vier Retires verteilen sich auf drei Takte. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 9 / 67

Agenda

1. Grenzen des Pipelining
2. ILP und Superskalarität
3. Out-of-Order Execution
4. Branch Prediction
5. Spekulation und Security (Spectre/Meltdown)
6. Labor-Ausblick (Compiler / Godbolt)

Notizen

Agenda knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Die Agenda fragt nach unabhängigen Befehlen, passenden Ressourcen, korrektem Zustand, Fetch am Branch, bleibenden Spuren und dem Compiler. Ein Laborlauf liegt nicht vor. Der ROB kommt vor dem Seitenkanal, weil er Recovery erklärt. Verständnischeck: Warum der ROB vor Security? Er trennt Architekturzustand und interne Spuren. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 10 / 67

Lernziele Block 7

- Skalare vs. superskalare Pipeline erklären
- Datenabhängigkeiten RAW, WAW, WAR unterscheiden
- Register Renaming und Reorder Buffer (ROB) einordnen
- Warum Branches tiefe Pipelines treffen
- Rolle von Compiler-Scheduling ($-O3$)

Notizen

Lernziele Block 7 knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Prüfbar sind Kanten, ein benanntes IPC-Fenster, der physische Quelltag, der präzise Exception-Zustand und

die Vorhersage vor dem Update. IPC 2 ohne Programm und Zählbasis ist unvollständig. -O3 ist kein garantierter Gewinn. Skalare vs. superskalare Pipeline erklären. Datenabhängigkeiten RAW, WAW, WAR unterscheiden. Verständnischeck: Welche Angaben fehlen bei IPC 2? Programm, Fenster und Zählbasis. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 11 / 67

Instruction-Level Parallelism (ILP)

- ILP: wie viele Befehle eines Programms *theoretisch* parallel laufen könnten
- Beispiel ohne Abhängigkeit:

```
a = b + c;
x = y * z;
```

- Unabhängige Operationen → auf getrennten Execution Units im selben Takt denkbar

Notizen

Instruction-Level Parallelism (ILP) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. ILP ist vorhandene Unabhängigkeit. Zwei C-Zeilen sind nicht automatisch zwei parallele ISA-Befehle. Pointer und Loads können weitere Kanten erzeugen. Units können verschiedene Latenzen haben. ILP: wie viele Befehle eines Programms *theoretisch* parallel laufen könnten. Beispiel ohne Abhängigkeit:. Verständnischeck: Sind zwei C-Zeilen automatisch parallel? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 12 / 67

Datenabhängigkeiten: RAW

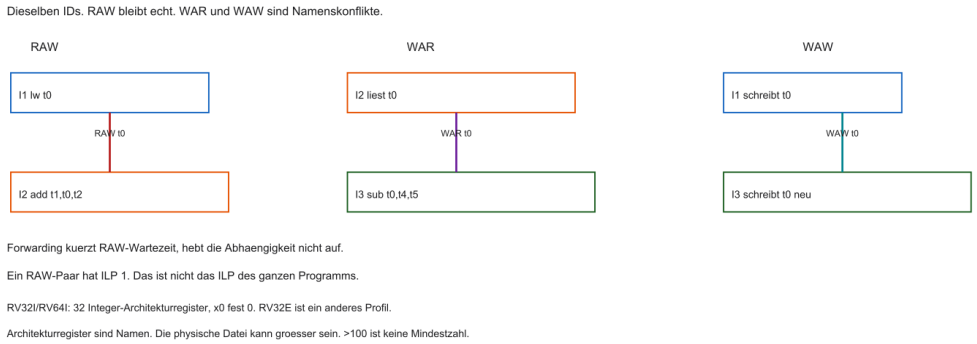


Abbildung 3: I1–I3: RAW bleibt, WAR und WAW sind Namenskonflikte auf t0.

- True Dependency / Read-After-Write (RAW): Befehl 2 braucht Ergebnis von Befehl 1

```
add t0, t1, t2    # schreibt t0
sub t3, t0, t4    # liest t0 – muss warten
```

- Dieses Paar hat ILP 1. Das ist keine Aussage über das ganze Programm
- Forwarding verkürzt die Wartezeit und hebt die RAW-Kante nicht auf

Notizen

Datenabhängigkeiten: RAW knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. RAW heißt Read After Write. Der Verbraucher braucht das Ergebnis des Produzenten. Forwarding liefert es früher, erzeugt es aber nicht vor der Rechnung. Ein RAW-Paar hat keine gleichzeitige unabhängige Ausführung. WAR und WAW sind Namenskonflikte und folgen später. True Dependency / Read-After-Write (RAW): Befehl 2 braucht Ergebnis von Befehl 1. Dieses Paar hat ILP 1. Das ist keine Aussage über das ganze Programm. Verständnischeck: Löscht Forwarding die Abhängigkeit? Nein, es kann nur die Wartezeit kürzen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 13 / 67

Statisches vs. dynamisches Scheduling

- Statisch (Compiler): Befehle vorab umordnen, Abhängigkeiten auseinanderziehen
- Dynamisch (Hardware): zur Laufzeit unabhängige Befehle finden und an die Units ausgeben (Issue)
- Compiler kennt u. a. Cache-Misses zur Laufzeit nicht → dynamisches Scheduling oft mächtiger

Notizen

Statisches vs. dynamisches Scheduling knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Statisches Scheduling legt die ISA-Reihenfolge vorher fest. Dynamisches Issue reagiert auf aktuelle Bereitschaft. Der Compiler kennt nicht jede Miss-Folge. Hardware sieht nur ein endliches Fenster. Beides ergänzt sich. Statisch (Compiler): Befehle vorab umordnen, Abhängigkeiten auseinanderziehen. Dynamisch (Hardware): zur Laufzeit unabhängige Befehle finden und an die Units ausgeben (Issue). Verständnischeck: Warum hilft dynamisches Scheduling bei wechselnden Loads? Es nutzt gerade bereite unabhängige Befehle. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 14 / 67

Superskalare Architekturen

- Skalar: typisch ein Issue pro Takt
- Superskalar: Multiple Issue. Das ist noch nicht Out-of-Order
- Fetch-Bytes, Decode, Issue, Ports und Retire-Breite sind verschiedene Größen
- Pipeline wird „breiter“: mehrspurige Autobahn statt einspurigem Fließband

Notizen

Superskalare Architekturen knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Superskalar heißt Multiple Issue und setzt OoO nicht voraus. Fetch, Decode, Issue und Retire können verschieden breit sein. Zwei ALU machen einen Kern nicht automatisch OoO. Ein Port bedient nur bestimmte Operationen. Skalar: typisch ein Issue pro Takt. Superskalar: Multiple Issue. Das ist noch nicht Out-of-Order. Verständnischeck: Sind zwei ALU automatisch OoO? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 15 / 67

Mehrere Execution Units

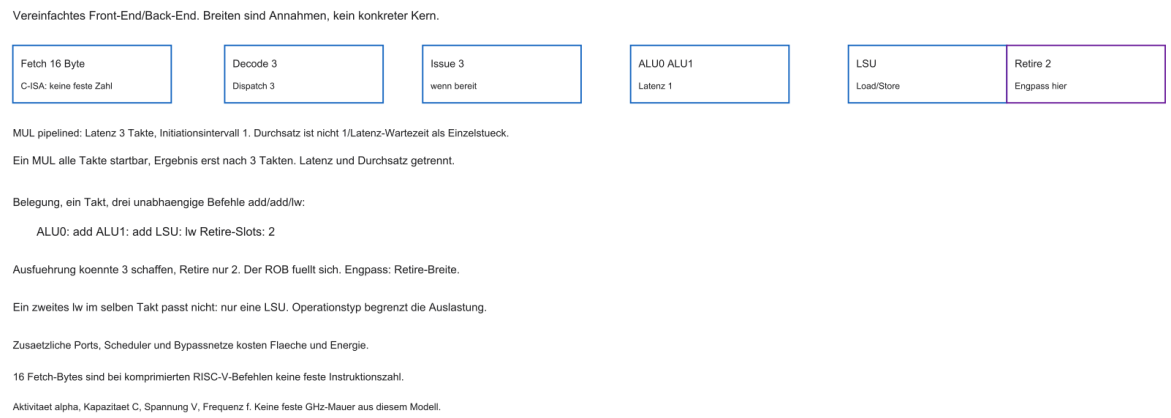


Abbildung 4: Fetch, Decode, Issue, zwei ALU, eine LSU, Retire 2. Der Engpass ist hier das Retire.

- Zwei Integer-ALU und eine LSU sind das vereinfachte Modell aus der Abbildung
- Ein pipelinierter Multiplizierer trennt Latenz und Initiationsintervall

Notizen

Mehrere Execution Units knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Das Ressourcenbild ist fiktiv: Fetch bis 16 Byte, Decode und Issue bis 3, zwei ALU, eine LSU, Retire bis 2. Drei Starts können den Retire-Engpass von 2 überholen, dann staut der Puffer. Ein Multiplizierer mit Latenz 3 und Intervall 1 startet unabhängige Operationen jeden Takt. MUL braucht die M-Erweiterung, nicht reines RV32I. Zwei Integer-ALU und eine LSU sind das vereinfachte Modell aus der Abbildung. Ein pipelinierter Multiplizierer trennt Latenz und Initiationsintervall. Verständnisccheck: Kann Latenz 3 trotzdem jeden Takt starten? Bei Intervall 1 ja, der Nutzer wartet trotzdem. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 16 / 67

Multi-Issue Fetch und Decode

- Fetch holt zum Beispiel 16 Byte. Mit der C-Erweiterung ist das keine feste Instruktionszahl
- Decode prüft Abhängigkeiten *zwischen* den Kandidaten und verteilt auf Units
- Aufwand der Abhängigkeitsprüfung wächst stark mit der Issue-Width

Notizen

Multi-Issue Fetch und Decode knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. 16 Fetch-Byte sind vier Befehle nur bei durchgängig 32-Bit-Instruktionen. Die C-Erweiterung meint

komprimierte Befehle, nicht die Sprache C. Decode erkennt Befehle. Nicht jede Bereitschaftsprüfung liegt im Decode. Fetch holt zum Beispiel 16 Byte. Mit der C-Erweiterung ist das keine feste Instruktionszahl. Decode prüft Abhängigkeiten *zwischen* den Kandidaten und verteilt auf Units. Verständnischeck: Sind 16 Byte stets vier Befehle? Nein, mit komprimierten Befehlen variiert die Zahl. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 17 / 67

In-Order Superskalar

- Start-Reihenfolge = Programmreihenfolge (z. B. ältere In-Order-Kerne)
- Unabhängige Nachbarn können gemeinsam starten
- Stoppt ein früher Befehl (Hazard/Miss), stauen dahinter auch Unabhängige

Notizen

In-Order Superskalar knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. In-order Issue wählt in Programmreihenfolge. Ein blockierter älterer Befehl kann jüngere Starts verhindern. Schon ausgegebene Befehle müssen nicht in derselben Reihenfolge fertig werden. Das globale Stoppen hinter dem Load ist die Lehrannahme von M3. Start-Reihenfolge = Programmreihenfolge (z. B. ältere In-Order-Kerne). Unabhängige Nachbarn können gemeinsam starten. Verständnischeck: Heißt In-Order gleiche Fertigreihenfolge? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Problem: Cache-Miss in-order

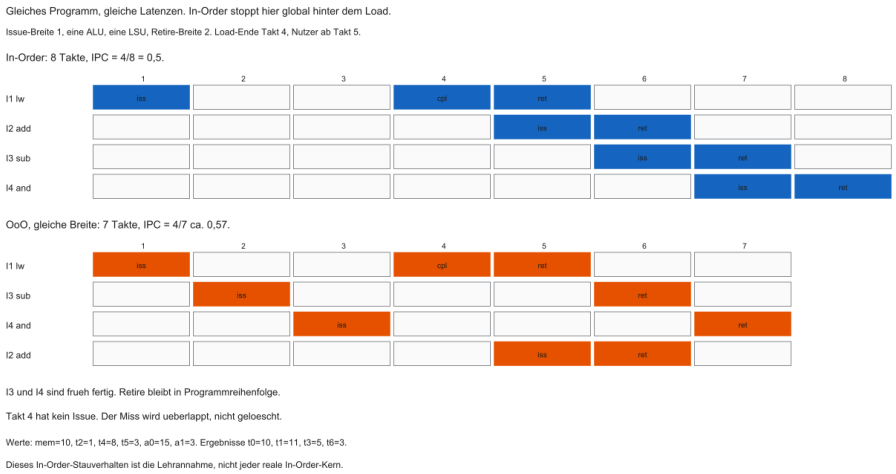


Abbildung 5: Gleiche Breite: in-order 8 Takte, OoO 7 Takte. Der Miss bleibt, Retire wartet.

```
lw  t0, 0(s1)      # I1, Ergebnis am Ende von Takt 4
add t1, t0, t2     # I2, RAW auf t0
sub t3, t4, t5     # I3, unabhängig
and t6, a0, a1     # I4, unabhängig
```

- Lehrannahme hier: der In-Order-Kern gibt hinter dem wartenden Load nichts weiter aus
- Reale In-Order-Kerne dürfen davon abweichen
- Der Miss bleibt auch bei OoO bestehen. Unabhängige Befehle können ihn nur überlappen

Notizen

Problem: Cache-Miss in-order knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. M3: lw t0, add t1, sub t3, and t6 mit a0 und a1. Speicher 10, t2 1, t4 8, t5 3, a0 15, a1 3. Ergebnisse 10, 11, 5 und 3. Nur I1 nach I2 ist RAW. Beide Varianten haben Issue 1, eine ALU, eine LSU und Retire 2. In-order acht Takte, IPC 0,5. OoO sieben Takte, IPC 4/7. I3 und I4 starten früh, retiren aber nach I2. Laufzeit sinkt um 12,5 Prozent, der Faktor ist 8/7. Lehrannahme hier: der In-Order-Kern gibt hinter dem wartenden Load nichts weiter aus. Reale In-Order-Kerne dürfen davon abweichen. Verständnischeck: Warum startet I2 erst in Takt 5? t0 ist erst am Ende von Takt 4 bereit. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 19 / 67

Falsche Abhängigkeit: WAW

Write-After-Write:

```
add t0, t1, t2   # schreibt t0
sub t0, t4, t5   # schreibt ebenfalls t0
```

- Keine RAW-Abhängigkeit der Operanden
- Der architektonisch bestätigte Endwert muss vom späteren Schreiber stammen
- Physische Abschlüsse dürfen nach Renaming überholen

Notizen

Falsche Abhängigkeit: WAW knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. WAW sind zwei Schreiber auf denselben ISA-Namen. Der spätere Schreiber bestimmt den architektonischen Endwert. Nach Renaming haben sie getrennte physische Ziele und dürfen fertig werden, bevor der Commit die richtige Version sichert. Keine RAW-Abhängigkeit der Operanden. Der architektonisch bestätigte Endwert muss vom späteren Schreiber stammen. Verständnischeck: Dürfen WAW-Schreiber in umgekehrter Reihenfolge fertig werden? Ja, mit getrennten Zielen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 20 / 67

Falsche Abhängigkeit: WAR

Write-After-Read:

```
add t1, t0, t2   # liest t0
sub t0, t4, t5   # überschreibt t0
```

- `sub` darf `t0` nicht zu früh überschreiben, bevor `add` gelesen hat
- Verhindert unkritische Überholvorgänge ohne weitere Maßnahmen

Notizen

Falsche Abhängigkeit: WAR knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. WAR: der ältere Leser braucht den alten Wert, der jüngere Schreiber erzeugt einen neuen. Der Leser speichert den alten physischen Quelltag. Das ist kein RAW, weil er das neue Ergebnis nicht braucht. `sub` darf `t0` nicht zu früh überschreiben, bevor `add` gelesen hat. Verhindert unkritische Überholvorgänge ohne weitere Maßnahmen. Verständnischeck: Warum bleibt der Leser korrekt? Sein Tag zeigt weiter auf die alte Version. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 21 / 67

Engpass: 32 ISA-Register

- RV32I und RV64I: 32 Integerregister `x0`–`x31`. `x0` ist fest 0
- Das sind Architektur-Namen, keine physischen Kisten. RV32E hat ein anderes Registerprofil
- Wiederverwendung erzeugt WAR/WAW. Erweiterungen bringen weitere Registerdateien

Notizen

Engpass: 32 ISA-Register knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. RV32I hat 32 Integernamen, x0 ist fest null, 31 sind beschreibbar. RV32 und RV64 sind die Bitbreite, nicht die Anzahl. t0 ist ein ABI-Alias, kein Extra-Register. Mehr physische Register ändern die sichtbare Anzahl nicht und lösen kein RAW. RV32I und RV64I: 32 Integerregister `x0`–`x31`. `x0` ist fest 0. Das sind Architektur-Namen, keine physischen Kisten. RV32E hat ein anderes Registerprofil. Verständnischeck: Wie viele beschreibbare Integerregister hat RV32I? 31. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 22 / 67

Aufgabe — RAW und Breite

Warum bleibt die Kante von I1 nach I2 bestehen, obwohl I3 und I4 unabhängig sind? Warum folgt aus zwei ALU nicht IPC 2?

Notizen

Aufgabe — RAW und Breite knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. I2 braucht t0 aus dem Load. I3 und I4 sind unabhängig, dürfen aber nicht vor I2 retiren. Zwei ALU garantieren weder zwei Starts noch zwei Retires. M3 nutzt bewusst eine ALU. IPC ist 4/8 und 4/7. Verständnischeck: Warum reicht die ALU-Zahl nicht für IPC? Programm, Fenster und Retire-Grenze fehlen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 23 / 67

Out-of-Order Execution (OoO)

- Dynamisches Scheduling: blockierte Befehle „parken“
- Unabhängige spätere Befehle dürfen vorgezogen werden
- Issue in datenabhängiger, dynamisch gewählter Reihenfolge. Der Kern bleibt getaktet

Notizen

Out-of-Order Execution (OoO) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. OoO ändert die Issue-Reihenfolge. Der PC holt weiter den vorhergesagten Strom. Rename und Dispatch bleiben geordnet, Retire auch. M3 zeigt OoO schon bei Issue-Breite eins. Dynamisches Scheduling: blockierte Befehle „parken“. Unabhängige spätere Befehle dürfen vorgezogen werden. Verständnischeck: Geht OoO mit Issue eins? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 24 / 67

Datenfluss-Maschine

- Instruktion „feuert“, wenn Operanden da sind — nicht nur wenn der PC sie erreicht
- Der Miss blockiert die Nutzer des Load. Unabhängige Befehle können vorbeilaufen, bis Issue-Queue oder ROB voll sind
- „Nur Abhängige warten“ gilt nicht ohne diese Grenzen

Notizen

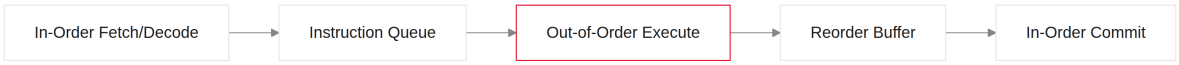
Datenfluss-Maschine knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Bereite Operanden genügen nicht. Unit, Port und Issue-Slot müssen passen. Ein voller ROB stoppt weitere Arbeit, auch wenn Befehle unabhängig sind. Instruktion „feuert“, wenn Operanden da sind — nicht nur wenn der PC sie erreicht. Der Miss blockiert die Nutzer des Load. Unabhängige Befehle können vorbeilaufen, bis Issue-Queue oder ROB voll sind. Verständnischeck: Reichen zwei bereite Operanden? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 25 / 67

Drei Phasen der OoO-Pipeline

1. In-Order Front-End: Fetch / Decode
2. Out-of-Order Back-End: Execute
3. In-Order Retire (Commit): Ergebnisse programmgerecht sichtbar machen

Dazwischen: große Puffer (Queues, ROB).



Notizen

Drei Phasen der OoO-Pipeline knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Vorne werden Befehle geholt, dekodiert und umbenannt. In der Mitte entstehen Werte in der PRF. Hinten bestätigt der ROB die Ordnung. Der Eintrag entsteht beim Dispatch, nicht erst wenn die ALU fertig ist. Der ROB speichert hier keinen Zahlwert. Verständnischeck: Entsteht der ROB-Eintrag erst bei Fertigstellung? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 26 / 67

Front-End und Renaming

- Mehrere Befehle aus dem I-Cache holen und dekodieren
- Vor dem Chaos: Register Renaming gegen WAW/WAR
- Architektur-Register der ISA ≠ physische Registerdatei

Notizen

Front-End und Renaming knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Das Front-End liest die aktuelle Map und reserviert ein neues Ziel. Ein jüngerer Befehl im selben Bündel sieht das gerade vergebene Ziel des älteren Produzenten. 32 Namen sind die Schnittstelle, keine Täuschung. Mehrere Befehle aus dem I-Cache holen und dekodieren. Vor dem Chaos: Register Renaming gegen WAW/WAR. Verständnischeck: Welche Version sieht der jüngere Befehl? Die neue physische Version des älteren Produzenten. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 27 / 67

Register Renaming

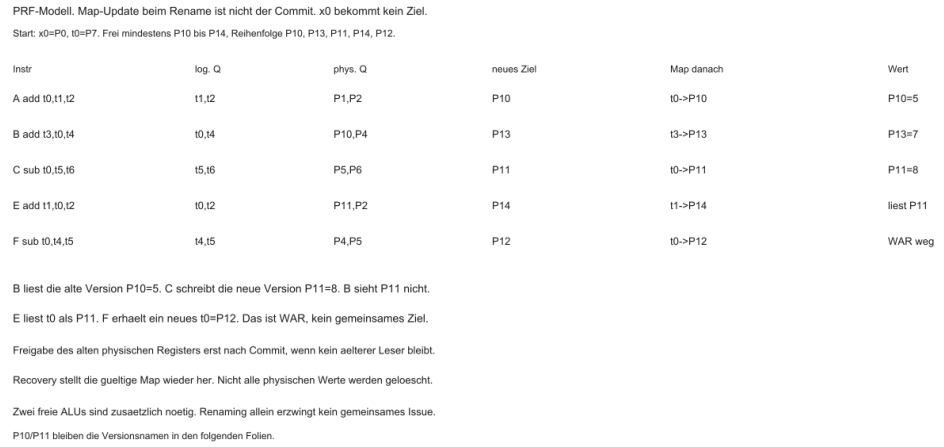
- Die ISA-Namen werden auf eine größere physische Datei abgebildet. Eine Zahl über 100 ist kein allgemeines Minimum
- Jeder Befehl mit Ziel außer der festen Null bekommt typisch ein neues physisches Register
- Befehle ohne Ziel und `x0` sind Ausnahmen

Notizen

Register Renaming knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Quellen zuerst lesen, altes Ziel merken, neues Ziel aus der Freiliste nehmen, Map aktualisieren, Ready löschen. x0 bekommt kein Ziel. Die alte Version bleibt, bis der Lebensdauervertrag sie freigibt. RAW bleibt als Tagabhängigkeit. Die ISA-Namen werden auf eine größere physische Datei abgebildet. Eine Zahl über 100 ist kein allgemeines Minimum. Jeder Befehl mit Ziel außer der festen Null bekommt typisch ein neues physisches Register. Verständnischeck: Warum bleibt P10 belegt, wenn t0 schon auf P11 zeigt? Ein älterer Leser kann P10 noch brauchen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 28 / 67

Renaming am Beispiel (WAW)



PRF-Modell. Map-Update beim Rename ist nicht der Commit. x0 bekommt kein Ziel.

Start: x0=P0, t0=P7. Frei mindestens P10 bis P14, Reihenfolge P10, P13, P11, P14, P12.

Instr	log. Q	phys. Q	neues Ziel	Map danach	Wert
A add t0,t1,t2	t1,t2	P1,P2	P10	t0->P10	P10=5
B add t3,t0,t4	t0,t4	P10,P4	P13	t3->P13	P13=7
C sub t0,t5,t6	t5,t6	P5,P6	P11	t0->P11	P11=8
E add t1,t0,t2	t0,t2	P11,P2	P14	t1->P14	liest P11
F sub t0,t4,t5	t4,t5	P4,P5	P12	t0->P12	WAR weg

B liest die alte Version P10=5. C schreibt die neue Version P11=8. B sieht P11 nicht.

E liest t0 als P11. F erhaelt ein neues t0=P12. Das ist WAR, kein gemeinsames Ziel.

Freigabe des alten physischen Registers erst nach Commit, wenn kein aelterer Leser bleibt.

Recovery stellt die gueltige Map wieder her. Nicht alle physischen Werte werden geloescht.

Zwei freie ALUs sind zusaetzlich noetig. Renaming allein erzwingt kein gemeinsames Issue.

P10/P11 bleiben die Versionsnamen in den folgenden Folien.

Abbildung 6: A bis E: Ziele P10, P13, P11, P14, P12. B liest P10, D liest P11.

```
# Lehrfolge A–E, nicht ausgeführt
add t0, t1, t2   # A, Ziel P10
add t3, t0, t4   # B, Quelle P10
sub t0, t5, t6   # C, Ziel P11
add t1, t0, t2   # D, Quelle P11
sub t0, t4, t5   # E, Ziel P12
```

Notizen

Renaming am Beispiel (WAW) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. A bis E schreiben P10, P13, P11, P14 und P12. Start: x0 ist P0, t0 ist P7. Frei sind mindestens P10 bis P14 in der Reihenfolge P10, P13, P11, P14, P12. Ergebnisse später 5, 7, 8, 9 und minus 7. B liest P10 gleich 5, nicht P11 gleich 8. D liest P11. A, C und E sind WAW, A nach B und C nach D sind RAW. Verständnischeck: Liest B den Wert 8? Nein, B hat P10 gespeichert. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 29 / 67

Reservation Stations

- Nach Renaming: Befehle in Reservation Stations / Issue Queue
- Klassisch Reservation Stations, alternativ eine gemeinsame Issue Queue. Ein einzelner Common Data Bus ist nur das Lehrbild
- Operand bereit reicht nicht: Unit, Port und Auswahlregel müssen passen

Notizen

Reservation Stations knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Eine Reservation Station hält Operation, Quelltags, Ready, Zieltag und Alter. Verteilte Stationen und eine gemeinsame Queue sind Varianten. Zwei bereite Adds und eine ALU starten nicht zugleich. Nach Renaming: Befehle in Reservation Stations / Issue Queue. Klassisch Reservation Stations, alternativ eine gemeinsame Issue Queue. Ein einzelner Common Data Bus ist nur das Lehrbild. Verständnischeck: Was verhindert den Doppeltstart? Die eine ALU und das Issue-Budget. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 30 / 67

Out-of-Order Issue

- Ergebnisse erscheinen auf dem Common Data Bus / Wakeup-Netz
- Wartende Einträge greifen Werte ab; bereite Befehle melden sich
- Scheduler gibt bereite Befehle in freie Units — Reihenfolge $\neq$ Programmzeile

Notizen

Out-of-Order Issue knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Completion markiert den Zieltag bereit. Der Scheduler wählt passende Kandidaten. Der Wert kommt aus der PRF oder einem Bypass, der Status geht an den ROB. Nicht jeder geweckte Nutzer startet sofort. Ergebnisse erscheinen auf dem Common Data Bus / Wakeup-Netz. Wartende Einträge greifen Werte ab; bereite Befehle melden sich. Verständnischeck: Starten alle Nutzer sofort? Nur wenn sie ausgewählt sind und eine Unit frei ist. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 31 / 67

Problem: Interrupts im Chaos

- Frühe Befehle noch nicht fertig, spätere schon executed
- Interrupt / Exception: architektonisch sichtbarer Zustand muss konsistent sein
- Ohne Schutz: RAM/Register könnten „Zukunft“ schon enthalten — inkonsistent für OS

Notizen

Problem: Interrupts im Chaos knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Ein jüngerer PRF-Wert ist noch kein Architekturzustand. Der Handler sieht die committed Map. Eine Exception gehört zu einem Befehl, ein Interrupt ist asynchron. Präzise heißt passende Programmsicht, keine feste Latenz. Frühe Befehle noch nicht fertig, spätere schon executed. Interrupt / Exception: architektonisch sichtbarer Zustand muss konsistent sein. Verständnischeck: Ist ein fertiger PRF-Wert für den Handler sichtbar? Nur wenn er committed ist. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 32 / 67

Reorder Buffer (ROB)

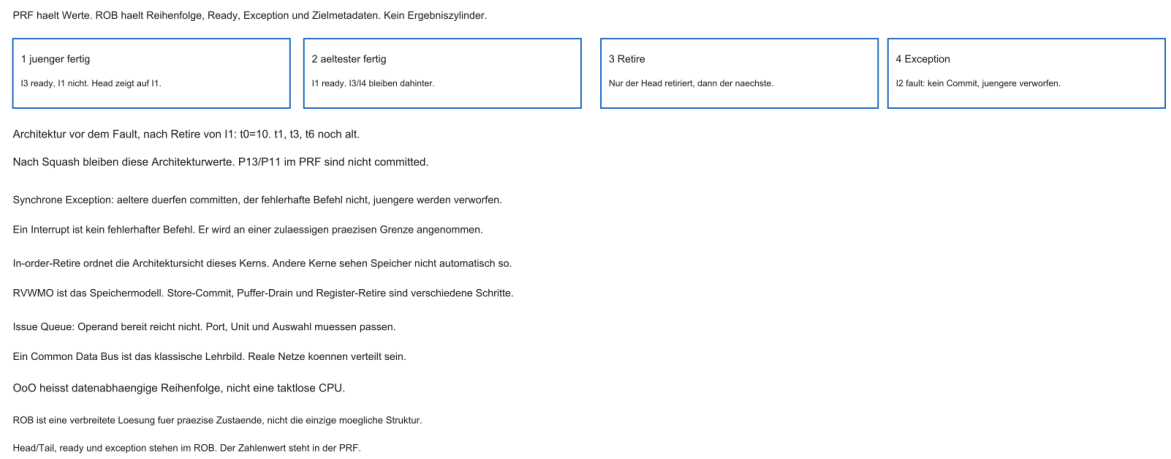


Abbildung 7: Werte stehen in der PRF. Der ROB merkt sich Ordnung, Ready und Exceptions.

- In diesem PRF-Modell schreibt die ALU in ein physisches Register, nicht in den ROB
- Der ROB führt Reihenfolge, Completion, Exception und Zielmetadaten. Ein result-holding ROB wäre ein anderes, hier nicht verwendetes Modell

Fertige jüngere Befehle stehen in physischen Registern, bis der Head retiriert.

Notizen

Reorder Buffer (ROB) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Im Modell hält die PRF den Zahlwert, der ROB Ordnung, Fertigstatus und Zielmetadaten. Die Exception-Variante ersetzt I2 durch lw t1, 0(s2) mit Zugriffsfehler. I1 hat t0 gleich 10 retired. I2 committet nicht. I3 und I4 werden verworfen. Ihre physischen Werte können bleiben, gelten aber nicht. In diesem PRF-Modell schreibt die ALU in ein physisches Register, nicht in den ROB. Der ROB führt Reihenfolge, Completion, Exception und Zielmetadaten. Ein result-holding ROB wäre ein anderes, hier nicht verwendetes Modell. Verständnischeck: Was speichert der ROB über ein add? Status und Metadaten, nicht den Zahlwert. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 33 / 67

In-Order Commit (Retire)

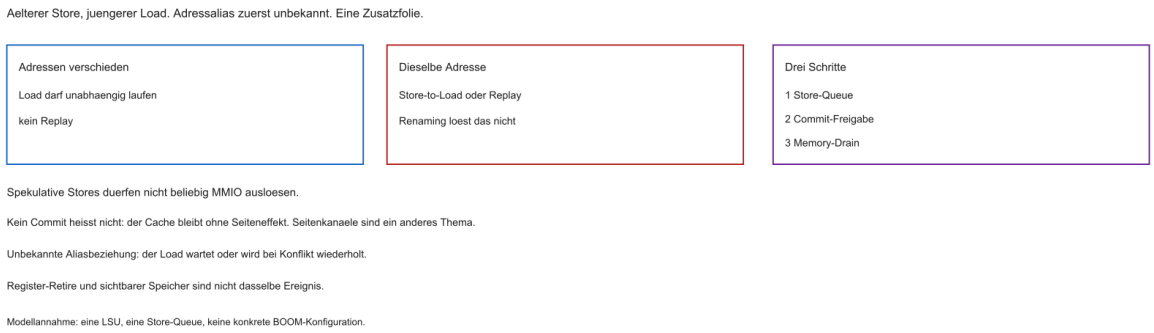


Abbildung 8: Store-Queue, Commit und Drain sind drei Schritte. Aliase kann Renaming nicht lösen.

- Retire nur am ältesten noch nicht retired Befehl, bis zur Retire-Breite
- Dadurch wird der Architektur-Registerzustand sichtbar. Ein Store darf danach noch im Puffer stehen
- Andere Beobachter sehen Speicher nach RVWMO, nicht automatisch in dieser Retire-Reihenfolge
- Spätere fertige Einträge warten, auch wenn ihre PRF-Werte schon stehen

Notizen

In-Order Commit (Retire) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Ein Store kann in der Queue vorbereitet, beim Commit freigegeben und erst danach geschrieben werden. Andere Kerne sehen ihn nach dem Speichermodell, nicht automatisch in Retire-Reihenfolge. Gleiche Adresse braucht Weiterleitung oder Warten. Unbekannte Adresse kann warten oder später wiederholt werden. MMIO ist kein normaler zurückrollbarer Wert. Retire nur am ältesten noch nicht retired Befehl, bis zur Retire-Breite. Dadurch wird der Architektur-Registerzustand sichtbar. Ein Store darf danach noch im Puffer stehen. Verständnischeck: Ist ein committed Store sofort für alle Kerne sichtbar? Nicht zwingend. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und

wurde nicht kompiliert oder ausgeführt.

Folie 34 / 67

Exceptions und ROB

- Synchrone Exception: ältere Instruktionen dürfen committen. Die fehlerhafte Instruktion committet ihren normalen Effekt nicht. Jüngere Effekte werden verworfen
- Ein asynchroner Interrupt hat keinen fehlerhaften Befehl. Er wird an einer zulässigen präzisen Grenze angenommen
- Recovery stellt das gültige Mapping wieder her

Notizen

Exceptions und ROB knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Beim Load-Fehler bleibt der Zustand vor I2: t0 gleich 10, die anderen Ziele alt. Branch-Recovery betrifft den falschen jüngeren Pfad. Exception-Recovery geht zum committed Zustand. Cachezeilen werden nicht automatisch gelöscht. Synchrone Exception: ältere Instruktionen dürfen committen. Die fehlerhafte Instruktion committet ihren normalen Effekt nicht. Jüngere Effekte werden verworfen. Ein asynchroner Interrupt hat keinen fehlerhaften Befehl. Er wird an einer zulässigen präzisen Grenze angenommen. Verständnischeck: Was heißt präzise Exception hier? Der Handler sieht den Zustand unmittelbar vor I2. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 35 / 67

Grenzen von OoO

Warum viele Cortex-M / MCU-Kerne ohne OoO bleiben:

- Transistoren: Renaming, Stations, ROB sind teuer
- Mehr Puffer und Vergleichsnetze kosten Fläche und Schaltaktivität. Watt statt Milliwatt ist keine allgemeine Grenze
- Determinismus: Laufzeiten schwerer vorherzusagen (Hard Real-Time)

Notizen

Grenzen von OoO knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Größere Dateien, Ports und Puffer kosten Fläche und Energie. Watt statt Milliwatt ist keine Familienregel. Eine einfache Pipeline ist mit Caches nicht automatisch taktfest. OoO erschwert Worst-Case-Zeiten, macht harte Echtzeit nicht unmöglich. Transistoren: Renaming, Stations, ROB sind teuer. Mehr Puffer und Vergleichsnetze kosten Fläche und Schaltaktivität. Watt statt Milliwatt ist keine allgemeine Grenze. Verständnischeck: Garantiert die Fünf-Stufen-Pipeline feste Laufzeit? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 36 / 67

Drei Phasen — Kurzfazit

Phase	Ordnung	Aufgabe
Fetch/Decode	in-order	lesen, renaming
Execute	out-of-order	Units auslasten
Commit	in-order	sichtbar machen, Exceptions

Notizen

Drei Phasen — Kurzfazit knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Geordnetes Umbenennen, dynamisches Issue, geordnetes Commit. Die Ausführungsreihenfolge darf vom Programm abweichen. Die sichtbare Bestätigung nicht. Verständnischeck: Welche Reihenfolge darf abweichen? Vor allem Issue und Abschluss, nicht das Commit. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 37 / 67

Aufgabe — Version und Exception

Welche physische Version liest Befehl B? Was bleibt architektonisch gültig, wenn I2 eine synchrone Exception auslöst, nachdem nur I1 retiriert wurde?

Notizen

Aufgabe — Version und Exception knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Zwei Beispiele: B liest P10 gleich 5, obwohl C schon P11 gleich 8 erzeugt hat. In der Exception-Variante ist nur I1 committed, t0 gleich 10. I2 ist der fehlerhafte Load, nicht das normale add. Die PRF muss nicht auf null gesetzt werden. Verständnischeck: Muss die PRF gelöscht werden? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 38 / 67

Control Hazards

- WAR und WAW kann Renaming entfernen. RAW bleibt. Kontrollfluss ist ein eigener Engpass
- Neuer Engpass: Kontrollfluss (`beq` und andere Branches)
- Beim Fetch des Branch: welcher nächste PC — Fall-through oder Target?
- Entscheidung oft erst nach dem Vergleich in EX (oder später in tiefen Pipelines)

Notizen

Control Hazards knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. beq vergleicht zwei Register. Das Ziel ist PC plus Offset. Bei 32-Bit-Befehlen ist der Geradeausweg PC plus 4. Fetch braucht Richtung und Ziel, bevor der Vergleich fertig ist. Renaming beseitigt diese Kontrollabhängigkeit nicht. WAR und WAW kann Renaming entfernen. RAW bleibt. Kontrollfluss ist ein eigener Engpass. Neuer Engpass: Kontrollfluss (`beq` und andere Branches). Verständnischeck: Warum eine Vorhersage? Der Fetch muss wählen, bevor der Vergleich fertig ist. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 39 / 67

Kosten eines Pipeline-Flush

- Warten auf die ALU: wenige Bubbles — bei 5 Stufen oft erträglich
- Moderne High-Performance-Kerne: deutlich tiefere Pipelines (Größenordnung: viele Stufen), u. a. für hohe Taktraten
- Falscher Weg → Flush vieler spekulativer Instruktionen
- Misprediction Penalty: oft im Bereich vieler Takte — teuer für IPC

Notizen

Kosten eines Pipeline-Flush knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Bei einer Fehlvorhersage werden jüngere Befehle des falschen Pfads verworfen. Ältere gültige Arbeit bleibt. Die Strafe hängt von Auflösung und Nachfüllen ab, nicht nur von der Stufenzahl. Pipeline-Flush ist nicht das Löschen einer Cachezeile. Warten auf die ALU: wenige Bubbles — bei 5 Stufen oft erträglich. Moderne High-Performance-Kerne: deutlich tiefere Pipelines (Größenordnung: viele Stufen), u. a. für hohe Taktraten. Verständnischeck: Wird der ganze Kern geleert? Nein, vor allem der falsche jüngere Pfad. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Misprediction: Zahlenbeispiel

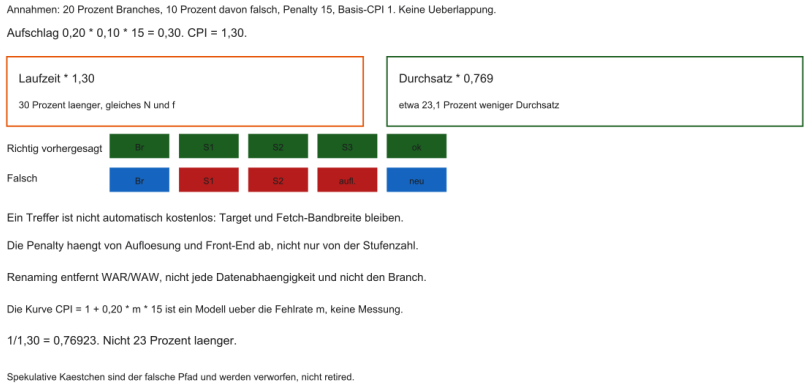


Abbildung 9: CPI 1,30 heißt 30 Prozent längere Laufzeit und etwa 23,1 Prozent weniger Durchsatz.

- $20\,\%$ der Befehle sind Branches
- Misprediction-Rate $10\,\%$, Penalty 15 Takte
- Basis-CPI $= 1$ (ideale Pipeline ohne andere Stalls)
- CPI-Aufschlag: $0{,}20 \times 0{,}10 \times 15 = 0{,}30$
- Effektives CPI $= 1{,}30$: bei gleichem N und f ist die Laufzeit 30 Prozent länger
- Der Durchsatz sinkt auf $1/1{,}30 \approx 0{,}769$, also rund 23,1 Prozent

Notizen

Misprediction: Zahlenbeispiel knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Basis-CPI 1, Branchanteil 0,20, Fehlrate 0,10 der Branches, Strafe 15 Takte. Zusatz 0,30, gesamt 1,30. 100 retired Befehle, 20 Branches, zwei Fehler, 30 Zusatztakte, 130 statt 100. Laufzeit plus 30 Prozent. Durchsatz 1/1,30 etwa 0,769, also etwa 23,1 Prozent weniger Durchsatz, nicht 23 Prozent längere Laufzeit. Falsche Pfade zählen nicht als N. $20\,\%$ der Befehle sind Branches. Misprediction-Rate $10\,\%$, Penalty 15 Takte. Verständnischeck: Warum 20 Prozent mal 10 Prozent? Das erste bezieht sich auf alle Befehle, das zweite nur auf Branches. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 41 / 67

Branch Prediction

- Nicht warten — spekulativ den wahrscheinlichen Weg füllen
- Vorhersage typisch schon beim Fetch
- Ein Treffer vermeidet den Flush, ist aber nicht automatisch kostenlos
- Fehlprognose: Flush / Misprediction Penalty
- Ziel: hohe Accuracy (in der Praxis oft deutlich über 90 %)

Notizen

Branch Prediction knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Eine Vorhersage ist ein früher Kandidat für den nächsten PC. Eine Trefferquote beschreibt Branches, nicht IPC. 90 Prozent sind hier keine garantierte Eigenschaft. Nicht warten — spekulativ den wahrscheinlichen Weg füllen. Vorhersage typisch schon beim Fetch. Verständnischeck: Beweist 90 Prozent Branchtreffer IPC 0,9? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 42 / 67

Statische Vorhersage

Einfache fest verdrahtete Heuristiken:

- Rückwärtssprung oft „taken“ (typisch Schleifenende)
- Vorwärtssprung oft „not taken“ (häufig seltene Fehlerpfade)

Hilfreich, aber für sehr tiefe Pipelines oft zu ungenau.

Notizen

Statische Vorhersage knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Rückwärts oft taken und vorwärts oft nicht genommen ist eine Heuristik, kein Gesetz. Ein Vorwärtssprung kann häufig genommen werden. Eine pauschale 90-Prozent- und 15-Stufen-Grenze gilt nicht. Rückwärtssprung oft „taken“ (typisch Schleifenende). Vorwärtssprung oft „not taken“ (häufig seltene Fehlerpfade). Verständnischeck: Kann ein Vorwärtssprung oft genommen werden? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 43 / 67

Dynamische Vorhersage

- Lernen zur Laufzeit aus der Branch-Historie
- Muster: was zuletzt passierte, passiert oft wieder
- Lange Schleifen: nach wenigen Iterationen fast immer korrektes „taken“

Notizen

Dynamische Vorhersage knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Der Zustand wird gelesen, die Richtung gesagt, danach mit dem Ergebnis aktualisiert. Ein- und Zweibitregeln sind feste Automaten, kein allgemeines Lernen. Der Prädiktor kennt das Ergebnis beim Raten noch nicht. Lernen zur Laufzeit aus der Branch-Historie. Muster: was zuletzt passierte, passiert oft wieder. Verständnischeck: Kennt er das Ergebnis schon? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 44 / 67

Branch History Table (BHT)

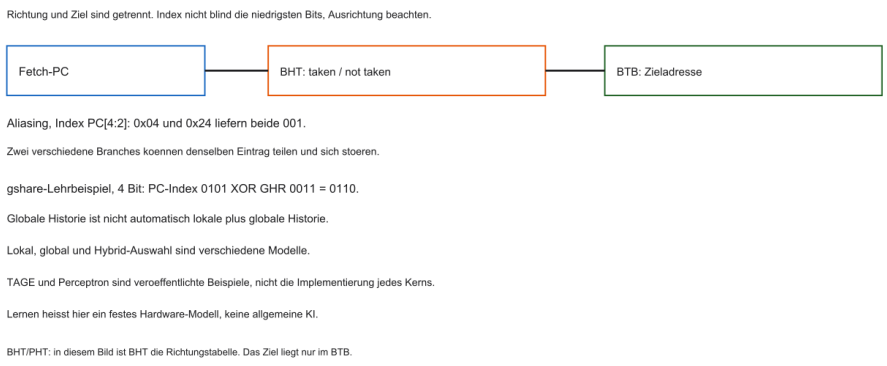


Abbildung 10: Richtung in der BHT, Ziel im BTB. 0x04 und 0x24 aliasen. gshare: 0101 XOR 0011 = 0110.

- Kleiner schneller Speicher (oft SRAM) für Predictions
- Index: typisch untere PC-Bits der Branch-Adresse
- Parallel zum Fetch: Eintrag liefert Taken/Not-Taken-Historie

Notizen

Branch History Table (BHT) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. BHT oder PHT hält den Richtungszustand, der BTB ein Ziel. Beide werden vom PC gespeist, nicht zwingend seriell. Bei acht Einträgen und festen 32-Bit-Befehlen ist der Index PC Bits 4 bis 2. 0x04 und

0x24 ergeben beide Index 1. Mit komprimierten Befehlen darf Bit 1 nicht pauschal entfallen. Aliasing ist hier eine Tabellenkollision. Kleiner schneller Speicher (oft SRAM) für Predictions. Index: typisch untere PC-Bits der Branch-Adresse. Verständnischeck: Warum teilen 0x04 und 0x24 den Index? Ihre Bits 4 bis 2 sind beide 001. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 45 / 67

1-Bit-Prädiktor

- Ein Bit: zuletzt Taken (1) oder Not Taken (0)
- Schwachpunkt: verschachtelte Schleifen
- Schleifenende kippt auf Not Taken → nächster Eintritt oft sofort falsch geraten
- Zwei Fehlprognosen pro äußerem Durchlauf sind typisch

Notizen

1-Bit-Prädiktor knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Start T, Folge T T T T NT zweimal: Fehler an den Schritten 5, 6 und 10, also drei von zehn. Danach irrt das Einbitmodell zweimal je weiterem Fünferblock. Nach NT sagt es beim nächsten T zunächst falsch NT. Ein Bit: zuletzt Taken (1) oder Not Taken (0). Schwachpunkt: verschachtelte Schleifen. Verständnischeck: Was sagt es nach einem NT? Zunächst NT. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

2-Bit saturierender Zähler

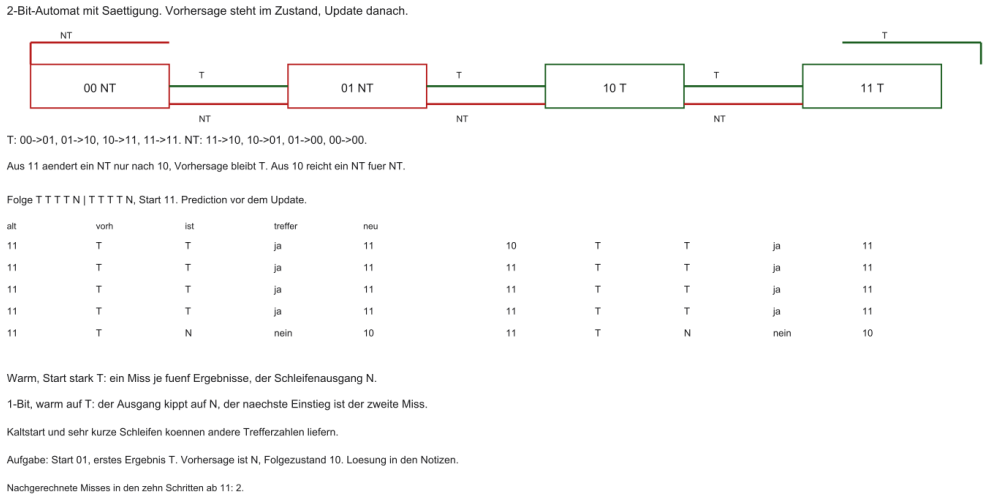


Abbildung 11: Vollständiger Automat. Ab Zustand 11 kostet TTTT N genau einen Miss je Durchlauf.

Industrie-Standard: Zustände 00–11

Code	Bedeutung	Vorhersage
00	Strongly Not Taken	Not Taken
01	Weakly Not Taken	Not Taken
10	Weakly Taken	Taken
11	Strongly Taken	Taken

Taken: $+1$ (Sättigung bei 11); Not Taken: -1 (Sättigung bei 00). Die Selbstschleifen an 00 und 11 stehen im Diagramm.

Notizen

2-Bit saturierender Zähler knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. 00 und 01 sagen NT, 10 und 11 sagen T. Der Zustand vor dem Update zählt. Start 11, dieselbe Zehnerfolge: Fehler bei 5 und 10. Start 01 und tatsächliches T: Vorhersage NT, danach Zustand 10. Das ist ein Lehrautomat, kein Industriegesetz. Verständnischeck: Welche Vorhersage gilt in 01 bei T? Zunächst NT, danach wird der Zustand 10. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 47 / 67

Hysterese des 2-Bit-Zählers

- Lange Schleife → oft 11 (Strongly Taken)
- Einmaliges Ende: $11 \rightarrow 10$ — Vorhersage bleibt Taken
- Nächster Schleifeneintritt: oft sofort korrekt; Zähler steigt wieder
- Aus einem starken Zustand braucht der Wechsel zwei widersprechende Ergebnisse. Aus einem schwachen Zustand genügt eines

Notizen

Hysterese des 2-Bit-Zählers knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Aus 11 führt ein NT nach 10, die Richtung bleibt T. Aus 10 führt ein NT nach 01, die Richtung wechselt sofort. Zwei Fehler bis zum Wechsel gelten nur für den starken Zustand. Lange Schleife → oft 11 (Strongly Taken). Einmaliges Ende: $11 \rightarrow 10$ — Vorhersage bleibt Taken. Verständnischeck: Warum kippt 10 schon bei einem NT? Der nächste Zustand 01 sagt bereits NT. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 48 / 67

Branch Target Buffer (BTB)

- Taken allein reicht nicht — Zieladresse muss früh bekannt sein
- Decode/Add für Offset oft zu spät für den nächsten Fetch
- BTB: Cache mit (u. a.) zuletzt beobachteter Target-Adresse
- IF: Prediction + bei Taken sofortiger Target aus dem BTB

Notizen

Branch Target Buffer (BTB) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Der BTB speichert Ziele erkannter Branches. Taken braucht die Adresse. Ein Fehltreffer im BTB kann kosten, obwohl die Richtung stimmte. Der Offset eines direkten beq ist nicht dasselbe wie ein indirektes Ziel. Taken allein reicht nicht — Zieladresse muss früh bekannt sein. Decode/Add für Offset oft zu spät für den nächsten Fetch. Verständnischeck: Kann die Richtung stimmen und der Fetch trotzdem warten? Ja, wenn das Ziel fehlt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 49 / 67

Globale Historie (Korrelation)

- Branches korrelieren oft im C-Kontrollfluss
- Global History Register (GHR): Bitmuster der letzten N Outcomes
- gshare ist ein konkretes Modell: PC-Index XOR globale Historie. Das ist nicht automatisch lokale plus globale Historie
- Erkennt Muster über einzelne Sprungstellen hinweg

Notizen

Globale Historie (Korrelation) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. gshare verknüpft hier den PC-Index 0101 mit der globalen Historie 0011 per XOR zu 0110. Das ist eine eigene Tabelle mit 16 Einträgen, nicht die Acht-Eintrag-Tabelle. Global ist die Historie verschiedener Branches, lokal die einer Stelle. Branches korrelieren oft im C-Kontrollfluss. Global History Register (GHR): Bitmuster der letzten N Outcomes. Verständnischeck: Kombiniert gshare lokal und global? Nein, hier PC und globale Historie. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 50 / 67

Fortgeschrittene Prädiktoren

- TAGE nutzt markierte Tabellen mit verschieden langen Historien
- Ein Perceptron gewichtet Historieneingaben und bildet eine Summe
- Das sind verschiedene veröffentlichte Ansätze, keine gemeinsame Gewichtungsregel

Notizen

Fortgeschrittene Prädiktoren knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. TAGE wählt unter markierten Tabellen mit verschieden langen Historien. Ein Perceptron gewichtet Eingaben und summiert. Das sind verschiedene Strukturen. Eine hohe Trefferquote wird nicht als Herstellerwert behauptet. TAGE nutzt markierte Tabellen mit verschieden langen Historien. Ein Perceptron gewichtet Historieneingaben und bildet eine Summe. Verständnischeck: Speichert TAGE dieselben Gewichte? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 51 / 67

Aufgabe — CPI und schwacher Zustand

Was bedeuten CPI 1,30 für Laufzeit und Durchsatz? Der 2-Bit-Zähler steht auf 01, das nächste Ergebnis ist T. Welche Vorhersage gilt?

Notizen

Aufgabe — CPI und schwacher Zustand knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. CPI 1,30 bedeutet bei gleichem N und f 30 Prozent mehr Laufzeit und etwa 23,1 Prozent weniger Durchsatz. Zustand 01 sagt NT. Das T ist die Fehlprognose, danach steht 10. Verständnischeck: Warum NT und nicht T? Die Vorhersage kommt aus 01, bevor das Update nach 10 geht. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 52 / 67

Spekulative Ausführung

- Spekulativer Fetch braucht kein OoO. Im High-Performance-Modell kommen Prediction und OoO zusammen
- Der vorhergesagte PC steuert zuerst Fetch. Issue folgt erst nach Decode, Rename und Bereitschaft
- ALUs rechnen und laden bereits — bevor der Branch architektonisch bestätigt ist

Notizen

Spekulative Ausführung knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Taken steuert zuerst den Fetch. Issue braucht Decode, Rename und eine freie passende Unit. Auflösung und Retire des Branches sind verschiedene Zeiten. Ein jüngerer falscher Befehl darf nicht committen. Spekulativer Fetch braucht kein OoO. Im High-Performance-Modell kommen Prediction und OoO zusammen. Der vorhergesagte PC steuert zuerst Fetch. Issue folgt erst nach Decode, Rename und Bereitschaft. Verständnischeck: Startet die ALU im Takt der Vorhersage? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 53 / 67

Rechnen auf Verdacht

- Bei korrekter Vorhersage kann das PRF-Ergebnis bereit sein; der ROB hält nur Status und Reihenfolge
- Speicherlatenzen werden hinter Spekulation versteckt
- IPC kann stark steigen, wenn Units nicht idle auf den Branch warten

Notizen

Rechnen auf Verdacht knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Das Zahlenergebnis liegt im physischen Register. Der ROB merkt Fertigstatus und Reihenfolge. Ein jüngeres add kann fertig sein, während der ältere Branch offen ist. Das garantiert keine feste IPC-Steigerung. Bei korrekter Vorhersage kann das PRF-Ergebnis bereit sein; der ROB hält nur Status und Reihenfolge. Speicherlatenzen werden hinter Spekulation versteckt. Verständnischeck: Wo liegt die Zahl? In der PRF, nicht als Wert im ROB. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 54 / 67

Rollback bei Misprediction

- Falscher Pfad: spekulative Loads/Rechnungen verwerfen
- Werte liegen in der PRF. Der ROB bestätigt sie erst in Programmreihenfolge
- Squash des spekulativen ROB-Teils, dann korrekten Pfad nachfüllen

Notizen

Rollback bei Misprediction knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Beim Fehlpfad bleiben ältere gültige Befehle. Jüngere Zuordnungen werden zurückgenommen. Ein Cache kann trotzdem verändert sein. Das ist kein Löschen der ganzen PRF und nicht dieselbe Recovery wie bei einer Exception. Falscher Pfad: spekulative Loads/Rechnungen verwerfen. Werte liegen in der PRF. Der ROB bestätigt sie erst in Programmreihenfolge. Verständnischeck: Warum reicht architektonisches Verwerfen nicht? Mikroarchitektonische Spuren können bleiben. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Spectre und Meltdown (2018)

- Lange Annahme: Fehlspekulation sei für Software unsichtbar
- Meltdown: spekulativ über Privilege-Grenzen (z. B. Kernel-Daten) — vor allem bestimmte ältere Designs
- Spectre: spekulativ über fehlvorhergesagte Branches / manipulierte Predictor-Zustände
- Gemeinsam: Architekturzustand wird zurückgerollt — Cache-/Timing-Spuren können bleiben

Notizen

Spectre und Meltdown (2018) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Spectre-v1 ist eine Fehlvorhersage eines erlaubten Kontrollflusses. Der ursprüngliche Meltdown betrifft bestimmte Implementierungen, in denen privilegierte Daten transient wirken, obwohl der Zugriff architektonisch scheitert. OoO allein beweist keine Verwundbarkeit jeder CPU. Lange Annahme: Fehlspekulation sei für Software unsichtbar. Meltdown: spekulativ über Privilege-Grenzen (z. B. Kernel-Daten) — vor allem bestimmte ältere Designs. Verständnischeck: Sind beide Mechanismen gleich? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 56 / 67

Seitenkanal über den Cache

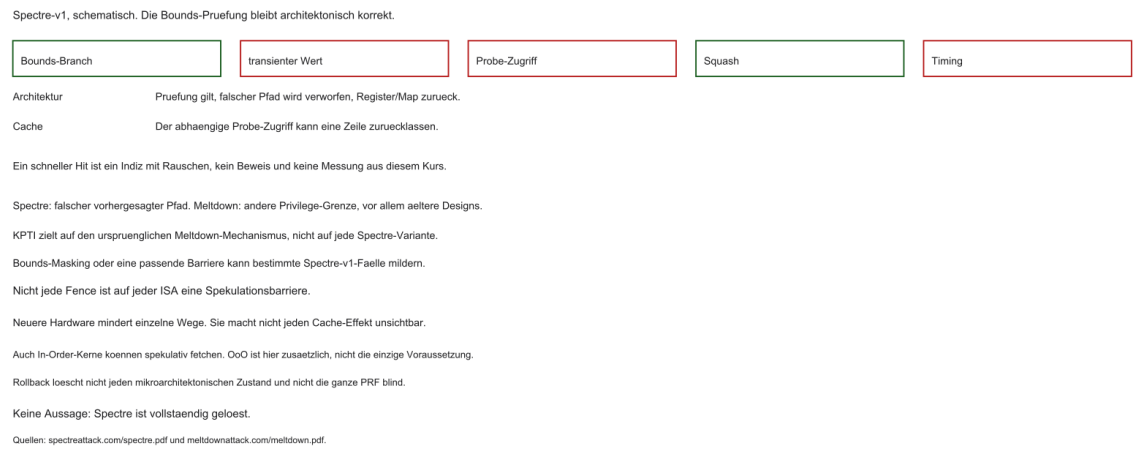


Abbildung 12: Die Prüfung bleibt korrekt. Der transiente Probe-Zugriff kann die Cache-Spur hinterlassen.

Prinzip (Spectre-artig, stark vereinfacht):

1. Die Bounds-Prüfung bleibt architektonisch korrekt
2. Der vorhergesagte Pfad läuft transient, bevor die Prüfung wirkt
3. Das Geheimnis steuert die Adresse eines folgenden Probe-Zugriffs
4. Squash verwirft den Architekturzustand. Die Cache-Spur kann bleiben

Notizen

Seitenkanal über den Cache knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Die Bounds-Prüfung bleibt architektonisch korrekt. Der falsche Pfad kann einen abhängigen Probe-Zugriff auslösen und eine Cachezeile hinterlassen. Das ist kein committed Registerwert. Es werden keine echten Zeiten oder Geheimnisse behauptet. Verständnischeck: Was kann bleiben? Eine Cachebelegung, kein committed Zielregister. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 57 / 67

Timing als Orakel

- Angreifer misst Zugriffszeiten (Flush+Reload / ähnliche Muster)
- Schneller Hit ⇒ Zeile war im Cache
- Aus der getroffenen Position lässt sich Information über spekulativ genutzte Werte rekonstruieren
- Klassischer mikroarchitektonischer Side Channel

Notizen

Timing als Orakel knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Ein schneller Zugriff ist ein verrauschtes Indiz. Prefetch und andere Zugriffe können denselben Effekt haben. Flush einer Cachezeile ist nicht der Pipeline-Flush. Ein einzelner Treffer beweist keinen geheimen Wert. Angreifer misst Zugriffszeiten (Flush+Reload / ähnliche Muster). Schneller Hit ⇒ Zeile war im Cache. Verständnischeck: Beweist ein schneller Zugriff den geheimen Wert? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 58 / 67

Mitigationen

- KPTI zielt vor allem auf den ursprünglichen Meltdown-Mechanismus
- Passendes Bounds-Masking oder eine geeignete Barriere kann bestimmte Spectre-v1-Fälle treffen. Nicht jede Fence ist auf jeder ISA eine Spekulationsbarriere
- Neuere Hardware mindert einzelne Wege und macht nicht jeden Cache-Effekt unsichtbar
- Spectre ist damit nicht vollständig gelöst

Notizen

Mitigationen knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. KPTI ist Kernel Page-Table Isolation und betrifft den ursprünglichen Meltdown-Fall, nicht generell Spectre-v1. Eine RISC-V-FENCE ordnet spezifizierte Zugriffe und ist keine allgemeine Spekulationsbarriere. Maßnahmen hängen von Variante und Plattform ab. KPTI zielt vor allem auf den ursprünglichen Meltdown-Mechanismus. Passendes Bounds-Masking oder eine geeignete Barriere kann bestimmte Spectre-v1-Fälle treffen. Nicht jede Fence ist auf jeder ISA eine Spekulationsbarriere. Verständnischeck: Warum ist KPTI keine Spectre-v1-Lösung? Es ändert Mappings, nicht jede Fehlvorhersage. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 59 / 67

Ausblick Labor: Compiler hilft der Pipeline

- Hardware kompensiert viel (OoO, BHT) — Compiler kann Hazards und Branches reduzieren
- GCC/Clang: Optimierungsstufen `-O0` … `-O3`
- Ziel: bessere IPC, weniger Stalls und teure Control Hazards

Notizen

Ausblick Labor: Compiler hilft der Pipeline knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Der Compiler formt die ISA-Folge. Er sieht ROB und Prädiktor nicht direkt. Weniger Befehle können trotz niedrigerem IPC schneller sein, weil N mal CPI kleiner wird. -O3 ist keine garantierte Beobachtung. Hardware kompensiert viel (OoO, BHT) — Compiler kann Hazards und Branches reduzieren. GCC/Clang: Optimierungsstufen `-O0` … `-O3`. Verständnischeck: Warum kann weniger IPC schneller sein? Weil N ausreichend sinken kann. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 60 / 67

Was `-O3` typisch tut

- `-O0` ist zum Debuggen gedacht und keine garantierte Zeile-für-Zeile-Übersetzung
- `-O2` und `-O3` erlauben mehr Analysen. `-O3` garantiert weder Unrolling noch weniger Branches noch kürzere Laufzeit
- Ob eine Schleife ausgerollt wird, hängt von Compiler, Version, `-march` und dem Code ab

Notizen

Was `-O3` typisch tut knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. -O0 ist keine zeilengetreue Übersetzung. -O3 erlaubt zusätzliche Umformungen, garantiert weder Unrolling noch kürzere Laufzeit. -march, -mtune und -mabi sind verschiedene Optionen. Ohne Toolchain gibt es hier keine Compilerausgabe. `-O0` ist zum Debuggen gedacht und keine garantierte Zeile-für-Zeile-Übersetzung. `-O2` und `-O3` erlauben mehr Analysen. `-O3` garantiert weder Unrolling noch weniger Branches noch kürzere Laufzeit. Verständnischeck: Beweisen weniger Assemblerzeilen kürzere Laufzeit? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 61 / 67

Trick: Instruction Scheduling

- Problem: `lw` → abhängiger Nutzer → Load-Use-Stall
- Compiler schiebt unabhängige Instruktionen in die Lücke (statisches Scheduling)
- C-Reihenfolge $\neq$ optimale Maschinenreihenfolge

Notizen

Trick: Instruction Scheduling knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Lehrreihenfolge, nicht GCC-Ausgabe: Load, Nutzer, unabhängige Subtraktion gegen Load, Subtraktion, Nutzer. Im kleinen Modell ist der Loadwert am Ende von Takt 2 da, der Nutzer frühestens in Takt 3. Die unabhängige Operation füllt Takt 2. Der Nutzer darf nicht vor den Load. Problem: `lw` → abhängiger Nutzer → Load-Use-Stall. Compiler schiebt unabhängige Instruktionen in die Lücke (statisches Scheduling). Verständnischeck: Warum nicht den Nutzer nach vorn? Das würde die RAW-Kante verletzen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 62 / 67

Trick: Loop Unrolling

- Enge Schleifen: viele Branches relativ zur Nutzarbeit
- Unrolling: Iterationen ausrollen → weniger `beq`, mehr gerader Code
- Trade-off: größere Code-Größe (I-Cache) vs. weniger Control Hazards

Notizen

Trick: Loop Unrolling knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Unrolling verringert Schleifenkontrolle. Ein einzelner Summenakkumulator behält seine RAW-Kette. Vier Teilsummen wären zusätzliche ILP und ein anderes Beispiel. uint32_t addiert modulo 2 hoch 32. Bei n gleich 5 verarbeitet der Viererschritt vier Elemente, der Rest eines. n gleich 0 greift nicht zu. Enge Schleifen: viele Branches relativ zur Nutzarbeit. Unrolling: Iterationen ausrollen → weniger `beq`, mehr gerader Code. Verständnischeck: Beseitigt vierfaches Unrolling die Summenabhängigkeit? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 63 / 67

Tool: Compiler Explorer (Godbolt)

- Web-Tool: C links, Assembler rechts (live)
- Farbzuordnung C-Zeile → Instruktionen
- Ideal, um `-O1` vs. `-O3` und RISC-V-Output zu vergleichen

Notizen

Tool: Compiler Explorer (Godbolt) knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Compiler Explorer zeigt gegebenenfalls Assembler, kein OoO-Timing. Farbe im Quelltext ist keine Eins-zu-eins-Abbildung. Ein Lauf liegt hier nicht vor. Web-Tool: C links, Assembler rechts (live). Farbzuordnung C-Zeile → Instruktionen. Verständnischeck: Was misst Godbolt nicht? Reale Stall- und Branch-Zyklen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 64 / 67

Laboridee: Godbolt und Messung

Geplant (Details im Übungsblatt):

- Godbolt zeigt die Übersetzung, kein OoO-Timing
- Ein Ripes-Vergleich gilt nur für das dort gewählte Pipeline-Modell, nicht für eine Desktop-OoO-CPU
- Renode ohne validiertes Mikroarchitekturmodell belegt keine Stall-, Branch- oder OoO-Zahlen
- Flags, Compilerversion und ABI gehören zur Beobachtung. Siehe Übungsblatt

Notizen

Laboridee: Godbolt und Messung knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Assembleransicht, ein Ripes-Modell und ein OoO-Zeitnachweis sind drei verschiedene Dinge. Renode ohne passendes Mikroarchitekturmodell belegt keine Stall-Zyklen. Ein korrektes Funktionsergebnis ist kein Speedup-Nachweis. Godbolt zeigt die Übersetzung, kein OoO-Timing. Ein Ripes-Vergleich gilt nur für das dort gewählte Pipeline-Modell, nicht für eine Desktop-OoO-CPU. Verständnischeck: Genügt ein funktionaler Simulator? Nein, für Zeit braucht es ein passendes Modell oder Hardware. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 65 / 67

Zusammenfassung Block 7

- Skalare Pipeline: Ideal oft IPC ≤ 1
- Superskalar: mehrere Units / Issues pro Takt
- OoO: Renaming + ROB für dynamisches Scheduling und präzise Exceptions
- Branch Prediction (u. a. 2-Bit, BTB): Flushes vermeiden
- Spekulation beschleunigt — Cache-Seitenkanäle (Spectre/Meltdown) als Kehrseite
- Compiler ($-O3$): Scheduling und Unrolling unterstützen die Hardware

Notizen

Zusammenfassung Block 7 knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Breite erlaubt passende Mehrfachstarts. OoO ändert die Auswahl. Renaming entfernt Namenskonflikte. Die PRF hält Werte, der ROB die Ordnung. Richtung und Ziel sind verschiedene Vorhersagen. Spekulation kann Cachespuren hinterlassen. -O3 garantiert kein Scheduling. Fertig muss man nachfragen: Abschluss oder Retire. Skalare Pipeline: Ideal oft IPC ≤ 1. Superskalar: mehrere Units / Issues pro Takt. Verständnischeck: Welche zwei Bedeutungen hat fertig? Ausführungsabschluss oder architektonisches Retire. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Folie 66 / 67

Evolution: MCT1 → MCT3

- MCT1: klare In-Order-Pipeline, begrenztes ILP
- MCT3 bisher: Caches, DMA, Bus-Mastering
- Heute: physische Register, OoO, Prediction, Spekulation
- Brücke zur realen High-Performance- und zur energieeffizienten MCU-Welt

Notizen

Evolution: MCT1 → MCT3 knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Unabhängigkeit kann schon im Programm stecken. OoO ändert die interne Durchführung, nicht die Bedeutung von add. Caches und DMA bleiben eigene Systempfade. MCU und Hochleistungskern sind verschiedene Abwägungen. MCT1: klare In-Order-Pipeline, begrenztes ILP. MCT3 bisher: Caches, DMA, Bus-Mastering. Verständnischeck: Ändert OoO die Bedeutung von add? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.

Ausblick und Abschluss

- Übungsblatt: Godbolt — `-O0`/`-O3`, Scheduling, Unrolling
- Optional: Zyklusvergleich in der Simulation

Vielen Dank — viel Erfolg beim Experimentieren.

Notizen

Ausblick und Abschluss knüpft an die bisherige Frage nach Ausführung und sichtbarem Zustand an. Die Zeitdiagramme zeigen Folgen der genannten Annahmen, keine gemessene Firmware. Geprüft sind Quellen, Semantik und synthetische Rechnungen. Nicht geprüft sind Kompilierung, Emulation und Seitenkanalmessung. Übungsblatt: Godbolt — `-O0`/`-O3`, Scheduling, Unrolling. Optional: Zyklusvergleich in der Simulation. Verständnischeck: Was wurde nachgewiesen? Die Konsequenz der Lehrannahmen, keine Messreihe. Der nächste Block setzt auf dieser ISA-Semantik auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert oder ausgeführt.
Wer Assembler-Optimierungen lesen kann, programmiert näher an der Hardware. Viel Erfolg!